1. Aim

To compare different methods of building a long monthly statement in Python, including repeated string concatenation using +=, collecting parts and using join(), and creating each line with an f-string. Their execution times and memory usage are analysed.

2. Problem Explanation

A mobile operator generates a monthly bill statement containing one line for each call or SMS. When a customer has many records, repeatedly adding strings can become slow because strings are immutable. This question compares different ways to build the same statement efficiently.

3. Prediction Before Execution

Before measuring, I predict that the list-and-join method will generally be faster for a large number of lines. Repeated += may create new strings and copy existing characters. The join() method combines the collected parts in one operation. An f-string is useful for formatting each individual line, but it does not by itself solve the problem of repeatedly combining the complete statement.

4. Python Program – Three String Building Methods

```
import time

def build_plus(parts):
    text = ""
    for p in parts:
        text += p + ";"
    return text


def build_join(parts):
    return "".join(p + ";" for p in parts)


def build_fstring(records):
    parts = []
    for i, amount in enumerate(records):
        parts.append(f"Item{i}:Rs{amount};")
    return "".join(parts)


# Create the same data for all three methods
items = [f"Item{i}:Rs{i % 50}" for i in range(200_000)]
amounts = [i % 50 for i in range(200_000)]

start = time.perf_counter()
a = build_plus(items)
t_plus = time.perf_counter() - start

start = time.perf_counter()
b = build_join(items)
t_join = time.perf_counter() - start

start = time.perf_counter()
c = build_fstring(amounts)
```

```
t_fstring = time.perf_counter() - start

print("plus and join same:", a == b)
print("plus and f-string same:", a == c)
print("length:", len(a))
print(f"+= time: {t_plus:.4f} s")
print(f"join time: {t_join:.4f} s")
print(f"f-string + join time: {t_fstring:.4f} s")
```

5. Complete the Code – TODO 1

The TODO can be completed with a single join() call. The separator is an empty string because the original program adds a semicolon after every item.

```
def build_join(parts):
    return "".join(p + ";" for p in parts)
```

This produces the same text as build_plus(parts), including the final semicolon.

6. Measurement at Different Input Sizes

The program below measures the two required methods at 1,000, 10,000 and 200,000 lines. Each test is run three times and the average is calculated. The values must be filled in using the results from the computer where the program is executed.

```
import time

def build_plus(parts):
    text = ""
    for p in parts:
        text += p + ";"
    return text

def build_join(parts):
    return "".join(p + ";" for p in parts)

def average_time(function, data, repeats=3):
    times = []
    for _ in range(repeats):
        start = time.perf_counter()
        result = function(data)
        times.append(time.perf_counter() - start)
    return sum(times) / len(times)

for n in (1_000, 10_000, 200_000):
    items = [f"Item{i}:Rs{i % 50}" for i in range(n)]
    plus_avg = average_time(build_plus, items)
    join_avg = average_time(build_join, items)
    print(f"{n:>7} lines | += {plus_avg:.6f}s | join {join_avg:.6f}s")
```

Number of lines	Average += time (seconds)	Average join time (seconds)
1,000	Record your result	Record your result
10,000	Record your result	Record your result
200,000	Record your result	Record your result

7. Explanation Using String Immutability

Python strings are immutable, which means that their contents cannot be changed after creation. In repeated concatenation, the interpreter may need to create a new string and copy the old contents together with the new part. Repeating this operation can involve copying many characters.

The join() method is recommended when many strings must be combined because it can calculate the required output size and combine the parts efficiently. It also makes the intention of joining many pieces clear. CPython may optimise some += operations in certain situations, so actual timings can vary; join() remains a predictable and clear approach for combining a collection of strings.

8. Alternative Concept – Writing to a File or Using StringIO

Instead of keeping the complete statement in memory, it can be written line by line to a file. This is useful when the statement is very large and does not need to remain as one in-memory string.

```
def write_statement(parts, filename):
    with open(filename, "w", encoding="utf-8") as file:
        for p in parts:
            file.write(p + ";")


from io import StringIO

def build_stringio(parts):
    buffer = StringIO()
    for p in parts:
        buffer.write(p + ";")
    return buffer.getvalue()
```

Writing to a file uses a small amount of working memory because each line can be written as it is generated. StringIO stores the statement in memory but provides an efficient file-like interface for building text.

9. Time and Space Complexity Analysis

Method	Time complexity	Space complexity	Explanation
Repeated += (general case)	Up to $O(N^2)$ character copying	O(N) final string	Each concatenation may copy the accumulated text again.
List followed by join()	O(N) in total output size	O(N)	Stores the parts and creates the final joined string.
f-string per line + join()	O(N) in total output size	O(N)	Formats each line and joins the completed pieces.
Write directly to file	O(N)	O(1) extra working memory (buffer aside)	Writes each part incrementally instead of storing the whole output.
StringIO	O(N)	O(N)	Builds text in an in-memory buffer.

Here, N represents the total number of output characters. For repeated +=, the worst-case copying cost can grow quadratically as the accumulated string becomes larger. The list-and-join approach takes linear time in the total output size.

10. Results and Analysis

After running the timing program three times for each input size, compare the average times. The 200,000-line test is especially important because it represents a heavy customer workload. Do not copy sample timings from another computer; record the actual values printed by your Python environment.

The expected result is that join() performs well as the number of lines increases. If += is close in speed for a particular test, this may be due to CPython optimisations and the size of the test. The measured values should be used to explain the actual result.

11. Execution Evidence

- Screenshot of the program showing that the generated texts are identical.
- Timing results for 1,000 lines.
- Timing results for 10,000 lines.
- Timing results for 200,000 lines.
- The three measured runs and average time for each method.

12. Conclusion

Three methods of building a monthly statement were considered. Repeated += is easy to write but may involve repeated copying of accumulated text. Building a list and joining it is a clear and efficient method for large statements. F-strings are useful for formatting individual lines. For very large statements that do not need to be held in memory, writing directly to a file is a suitable alternative. The final choice should be supported by the timing results collected on the computer.